

자연어 처리(NLP) — 토큰화와 임베딩 완전 개념 정리

5장 토큰화

1. 자연어 처리란?

자연어 (Natural Language)

자연 언어라고도 부르며, 인공적으로 만들어진 프로그래밍 언어와 다르게 사람들이 쓰는 언어 활동을 위해 자연히 만들어진 언어를 의미한다.

자연어 처리 (NLP, Natural Language Processing)

컴퓨터가 인간의 언어를 이해하고 해석 및 생성하기 위한 기술을 의미한다.

자연어 처리는 인공지능의 하위 분야 중 하나로, 컴퓨터가 인간과 유사한 방식으로 인간의 언어를 이해하고 처리하는 것이 주요 목표다. 인간 언어의 구조, 의미, 맥락을 분석하고 이해할 수 있는 알고리즘과 모델을 개발한다.

해결해야 할 문제

모호성 (Ambiguity)

인간의 언어는 단어와 구가 사용되는 맥락에 따라 여러 의미를 갖게 되어 모호한 경우가 많다. 알고리즘은 이러한 다양한 의미를 이해하고 명확하게 구분할 수 있어야 한다.

가변성 (Variability)

인간의 언어는 다양한 사투리(Dialects), 강세(Accent), 신조어(Coined word), 작문 스타일 등으로 인해 매우 가변적이다. 알고리즘은 이러한 가변성을 처리할 수 있어야 하며 사용 중인 언어를 이해할 수 있어야 한다.

구조 (Structure)

인간의 언어는 문장이나 구의 의미를 이해할 때 구문(Syntactic)을 파악하여 의미(Semantic)를 해석한다. 알고리즘은 문장의 구조와 문법적 요소를 이해하여 의미를 추론하거나 분석할 수 있어야 한다.

2. 말뭉치와 토큰

말뭉치 (Corpus)

자연어 모델을 훈련하고 평가하는 데 사용되는 대규모의 자연어를 뜻한다. 뉴스 기사, 사용자 리뷰, 저널이나 칼럼 등에서 목적에 따라 구축되는 텍스트 데이터를 의미한다. 말뭉치는 일련의 단어의 가능성을 예측하는 알고리즘인 언어 모델을 구축하고 평가하는 데 자주 사용된다.

토큰 (Token)

개별 단어나 문장 부호와 같은 텍스트를 의미하며 말뭉치보다 더 작은 단위다. 토큰은 텍스트의 개별 단어, 구두점 또는 기타 의미 단위일 수 있다. 토큰으로 나누는 목적은 컴퓨터가 자연어를 이해할 수 있게 나누는 과정이다.

토큰화 (Tokenization)

말뭉치를 토큰으로 나누는 과정. 컴퓨터가 텍스트를 보다 효율적으로 분석하고 처리할 수 있도록 하는 중요한 단계다.

토큰나이저 (Tokenizer)

텍스트 문자열을 토큰으로 나누는 알고리즘 또는 소프트웨어를 의미한다. 텍스트에서 발생하는 다양한 단어와 구문을 식별하고 분석할 수 있게 하므로 언어 모델링 또는 기계 번역과 같은 다양한 자연어 처리 작업에서 사용된다.

토큰나이저를 구축하는 방법

방법	설명
공백 분할	텍스트를 공백 단위로 분리해 개별 단어로 토큰화한다
정규표현식 적용	정규 표현식으로 특정 패턴을 식별해 텍스트를 분할한다
어휘 사전(Vocabulary) 적용	사전에 정의된 단어 집합을 토큰으로 사용한다
머신러닝 활용	데이터세트를 기반으로 토큰화하는 방법을 학습한 머신러닝을 적용한다

OOV (Out of Vocab)

어휘 사전 방법은 직접 어휘 사전을 구축하기 때문에 없는 단어나 토큰이 존재할 수 있다. 이러한 토큰을 OOV라고 한다. 어휘 사전 방법은 OOV 문제를 고려해 토큰화하는 것이 중요하다.

차원의 저주 (Curse of Dimensionality)

큰 어휘 사전을 구축하면 학습 비용의 증대는 물론이고, 자칫 차원의 저주에 빠질 수 있다. 모든 토큰이나 단어를 벡터화하면 어휘 사전에 등장하는 토큰 개수만큼의 차원이 필요하고, 벡터값이 거의 모두 0의 값을 가지는 **희소(sparse)** 데이터로 표현된다. 또한 희소 데이터의 표현 방법은 어휘 사전에 등장하는 출현 빈도만 고려하기 때문에 문장에서 발생한 토큰들의 순서 관계를 잘 표현하지 못한다.

3. 단어 및 글자 토큰화

토큰화는 자연어 처리에서 매우 중요한 전처리 과정으로, 텍스트 데이터를 구조적으로 분해하여 개별 토큰으로 나누는 작업을 의미한다. 이러한 토큰화 과정은 정확한 분석을 위해 필수이며, 단어나 문장의 빈도수, 출현 패턴 등을 파악할 수 있다.

입력된 텍스트 데이터를 단어(Word)나 글자(Character) 단위로 나누는 기법으로는 단어 토큰화와 글자 토큰화가 있다. 이러한 기법을 통해 각각의 토큰은 의미를 갖는 최소 단위로 분해된다.

단어 토큰화 (Word Tokenization)

자연어 처리 분야에서 핵심적인 전처리 작업 중 하나로 텍스트 데이터를 의미 있는 단위인 단어로 분리하는 작업이다.

띄어쓰기, 문장 부호, 대소문자 등의 특정 구분자를 활용해 토큰화가 수행된다. 품사 태깅, 개체명 인식, 기계 번역 등의 작업에서 널리 사용되며 가장 일반적인 토큰화 방법이다.

`split()` 메서드를 사용하며, 구분자를 입력하지 않으면 **공백(Whitespace)** 을 기준으로 나눈다.

단점

- 한국어 접사, 문장 부호, 오타 혹은 띄어쓰기 오류 등에 취약하다
- 예: `cg` 라는 토큰이 단어 사전에 있더라도 `cg.`, `cg는`, `cg도` 등은 OOV가 된다

글자 토큰화 (Character Tokenization)

띄어쓰기뿐만 아니라 글자 단위로 문장을 나누는 방식. 비교적 작은 단어 사전을 구축할 수 있다는 장점이 있다.

- 작은 단어 사전을 사용하면 학습 시 컴퓨터 자원을 아낄 수 있다
- 언어 모델링과 같은 시퀀스 예측 작업에서 활용된다
- 영어의 경우 각 알파벳으로 나뉜다
- 단어 토큰화와 다르게 공백도 토큰으로 나뉜다

단점

- 개별 토큰은 아무런 의미가 없으므로 자연어 모델이 각 토큰의 의미를 조합해 결과를 도출해야 한다
- 모델 입력 시퀀스의 길이가 길어질수록 연산량이 증가한다
- 다의어나 동음이의어가 많은 도메인에서 구별하는 것이 어려울 수 있다

자소 단위 토큰화

한글의 경우 하나의 글자는 여러 자음과 모음의 조합으로 이루어져 있어 **자소** 단위로 나눠서 토큰화를 수행한다.

자소란 언어의 문자 체계에서 의미상 구별할 수 있는 가장 작은 단위. 영어에서는 a, b, c, 한글에서는 ㄱ, ㄴ, ㅏ 등이 해당된다.

jamo 라이브러리를 활용하며, 한글 문자 및 자모 작업을 위한 한글 음절 분해 및 합성 라이브러리다.

한글 인코딩 방식

방식	설명
조합형	글자를 자모 단위로 나눠 인코딩한 뒤 이를 조합해 한글을 표현한다
완성형	조합된 글자 자체에 값을 부여해 인코딩하는 방식이다

jamo 주요 함수

함수	설명
<code>h2j</code>	완성형으로 입력된 한글을 조합형 한글로 변환한다. 입력된 한글 문자열을 유니코드 U+1100~U+11FE 사이의 조합형 한글 자모로 변환하는 함수다
<code>j2hcg</code>	조합형 한글 문자열을 자소 단위로 나눠 반환하는 함수다. 조합형 한글로 입력된 문자열은 초성, 중성, 종성으로 나뉜다

시퀀스 길이 비교

토큰화 방식	입력 시퀀스 길이
단어 토큰화	13
글자 토큰화	53
자소 단위 토큰화	101

4. 형태소 토큰화

형태소 (Morpheme)

실제로 의미를 가지고 있는 최소의 단위.

형태소 토큰화(Morpheme Tokenization) 란 텍스트를 형태소 단위로 나누는 토큰화 방법으로 언어의 문법과 구조를 고려해 단어를 분리하고 이를 의미 있는 단위로 분류하는 작업이다.

한국어와 같이 교착어(Agglutinative Language)인 언어에서 중요하게 수행된다. 한국어는 어근에 다양한 접사와 조사가 조합되어 하나의 낱말을 이루므로 각 형태소를 적절히 구분해 처리해야 한다.

예) '그는 나에게 인사를 했다'

→ 그는 = 그 (단어) + 는 (조사)

→ 나에게 = 나 (단어) + 에게 (조사)

형태소의 종류

종류	설명	예시
자립 형태소 (Free Morpheme)	스스로 의미를 가지고 있는 형태소. 명사, 동사, 형용사와 같은 단어를 이루는 기본 단위	그, 나, 인사
의존 형태소 (Bound Morpheme)	스스로 의미를 갖지 못하고 다른 형태소와 조합되어 사용되는 형태소. 조사, 어미, 접두사, 접미사 등	-는, -에게, -를, 했-, -다

형태소 어휘 사전 (Morpheme Vocabulary)

자연어 처리에서 사용되는 단어의 집합인 어휘 사전 중에서도 각 단어의 형태소 정보를 포함하는 사전을 말한다. 단어가 어떤 형태소들의 조합으로 이루어져 있는지에 대한 정보를 담고 있어, 형태소 분석 작업에서 매우 중요한 역할을 한다.

형태소 단위로 어휘 사전을 구축하면 새로운 단어도 기존 형태소 조합으로 쉽게 학습할 수 있어 OOV 문제를 줄일 수 있다.

일반적으로 형태소 어휘 사전에는 각 형태소가 어떤 품사에 속하는지와 해당 품사의 뜻 등의 정보도 함께 제공된다.

품사 태깅 (POS Tagging)

텍스트 데이터를 형태소 분석하여 각 형태소에 해당하는 **품사(Part Of Speech, POS)** 를 태깅하는 작업. 이를 통해 자연어 처리 분야에서 문맥을 고려할 수 있어 더욱 정확한 분석이 가능해진다.

KoNLPy

한국어 자연어 처리를 위해 개발된 라이브러리로 명사 추출, 형태소 분석, 품사 태깅 등의 기능을 제공한다. 텍스트 마이닝, 감성 분석, 토픽 모델링 등 다양한 NLP 작업에서 사용한다.

자바 언어 기반의 한국어 형태소 분석기로 **JDK(Java Development Kit)** 기반으로 개발됐다.

지원하는 형태소 분석기

분석기	설명
Okt (Open Korean Text)	SNS 텍스트 데이터를 기반으로 개발
꼬꼬마 (Kkma)	국립국어원에서 배포한 세종 말뭉치를 기반으로 학습
코모란 (Komoran)	-
한나눔 (Hannanum)	-
메캅 (Mecab)	윈도우 환경에서 지원되지 않으며 맥이나 우분투 같은 환경에서만 설치 가능

Okt 주요 메서드

메서드	설명
<code>okt.nouns</code>	입력된 문장에서 명사만 추출해 리스트를 반환한다
<code>okt.phrases</code>	입력된 문장에서 어절 구 단위를 추출해 리스트를 반환한다
<code>okt.morphs</code>	입력된 문장에서 형태소만 추출해 리스트를 반환한다
<code>okt.pos</code>	입력된 문장에서 각 단어에 대한 품사 정보를 추출하여 (형태소, 품사) 형태의 튜플로 구성된 리스트를 반환한다

꼬꼬마(Kkma) 주요 메서드

메서드	설명
<code>kkma.nouns</code>	명사 추출
<code>kkma.sentences</code>	문장 추출 (구문 추출 기능은 지원하지 않음)
<code>kkma.morphs</code>	형태소 추출
<code>kkma.pos</code>	품사 태깅

Okt vs 꼬꼬마 비교

항목	Okt	꼬꼬마
품사 수	19개	56개

항목	Okt	꼬꼬마
특징	SNS 기반, 빠름	세밀한 분석 가능, 느림

태깅하는 품사의 수가 많으면 더 자세한 단위로 분석이 가능하지만, 품사 태깅에 소요되는 시간도 길어지며, 더 많은 품사로 분리해 모델의 성능이 저하될 수도 있다. 시스템의 목적과 환경에 맞는 적절한 형태소 분석기를 사용해야 한다.

NLTK (Natural Language Toolkit)

자연어 처리를 위해 개발된 라이브러리로, 토큰화, 형태소 분석, 구문 분석, 개체명 인식, 감성 분석 등과 같은 기능을 제공한다.

주로 영어 자연어 처리를 위해 개발됐지만, 네덜란드어, 프랑스어, 독일어 등과 같은 다양한 언어의 자연어 처리를 위한 데이터와 모델을 제공한다.

사용 모델

모델	설명
Punkt	통계 기반 모델. 트리뱅크(Treebank)라는 대규모 영어 말뭉치를 기반으로 학습됨
Averaged Perceptron Tagger	퍼셉트론을 기반으로 품사 태깅을 수행. 총 35개의 품사를 태깅할 수 있음

주요 메서드

메서드	설명
<code>word_tokenize</code>	문장을 입력받아 공백을 기준으로 단어를 분리하고, 구두점 등을 처리해 각각의 단어(token)를 추출해 리스트로 반환한다
<code>sent_tokenize</code>	문장을 입력받아 마침표(.), 느낌표(!), 물음표(?) 등의 구두점을 기준으로 문장을 분리해 리스트로 반환한다
<code>pos_tag</code>	토큰화된 문장에서 품사 태깅을 수행한다. 토큰화된 단어들이 들어가야 하며 Averaged Perceptron Tagger 모델이 필요하다

spaCy

사이썬(Cython) 기반으로 개발된 오픈 소스 라이브러리로서, 자연어 처리를 위한 기능을 제공한다.

NLTK vs spaCy 비교

항목	NLTK	spaCy
목적	학습용, 다양한 알고리즘과 예제 제공	효율적인 처리 속도와 높은 정확도
특징	다양한 언어 지원	머신러닝 기반, GPU 가속 지원
리소스	상대적으로 적음	더 크고 복잡한 모델 필요

spaCy는 GPU 가속을 비롯해 영어, 프랑스어, 한국어, 일본어 등을 비롯해 24개 이상의 언어로 사전 학습된 모델을 제공한다.

spaCy 객체 구조

- **nlp** 인스턴스 에 문장을 입력하면 **doc** 객체 가 반환된다
- **doc** 객체 는 다시 여러 **token** 객체 로 이루어져 있다
- **token** 객체 의 속성을 통해 다양한 정보를 확인할 수 있다

token 객체 주요 속성

속성	설명
<code>pos_</code>	기본 품사 속성
<code>tag_</code>	세분화 품사 속성
<code>text</code>	원본 텍스트 데이터
<code>text_with_ws</code>	토큰 사이의 공백을 포함하는 텍스트 데이터
<code>vector</code>	벡터
<code>vector_norm</code>	벡터 노름

5. 하위 단어 토큰화 (Subword Tokenization)

형태소 분석기는 신조어, 외래어, 오타자, 고유어 등 모르는 단어를 적절한 단위로 나누는 것에 취약하다. 이는 어휘 사전의 크기를 크게 만들고 OOV에 대응하기 어렵게 만든다.

하위 단어 토큰화(Subword Tokenization) 란 하나의 단어가 빈번하게 사용되는 **하위 단어(Subword)** 의 조합으로 나누어 토큰화하는 방법이다.

예) `Reinforcement` → `Rein` + `force` + `ment`

장점

- 단어의 길이를 줄일 수 있어서 처리 속도가 빨라진다
- OOV 문제 완화
- 신조어, 은어, 고유어 등으로 인한 문제를 완화할 수 있다

하위 단어 토큰화 방법의 종류

- 바이트 페어 인코딩 (BPE)
- 워드피스 (Wordpiece)
- 유니그램 (Unigram)
- 바이트 수준 바이트 페어 인코딩 (BBPE)

바이트 페어 인코딩 (BPE, Byte Pair Encoding)

다이그램 코딩(Digram Coding)이라고도 한다. 텍스트 데이터에서 가장 빈번하게 등장하는 글자 쌍의 조합을 찾아 부호화하는 압축 알고리즘으로, 초기에는 데이터 압축을 위해 개발됐으나 자연어 처리 분야에서 하위 단어 토큰화를 위한 방법으로 사용된다.

동작 방식

1. 모든 단어를 글자 단위로 나눈다
2. 가장 빈도수가 높은 글자 쌍을 탐색한다
3. 해당 글자 쌍을 하나의 새 토큰으로 병합하고 어휘 사전에 추가한다
4. 연속된 글자 쌍이 더 이상 나타나지 않거나 정해진 어휘 사전 크기에 도달할 때까지 반복한다

자주 등장하는 단어는 하나의 토큰으로, 덜 등장하는 단어는 여러 토큰의 조합으로 표현된다.

예시 (abracadabra)

단계	결과	설명
원문	abracadabra	-
Step 1	AracadAra	'ab' 쌍을 'A'로 치환
Step 2	ABcadAB	'ra' 쌍을 'B'로 치환
Step 3	CcadC	'AB' 쌍을 'C'로 치환

토큰나이저로서 BPE는 자주 등장하는 글자 쌍을 찾아 치환하는 대신 어휘 사전에 추가한다.

BPE 알고리즘을 이용해 어휘 사전을 구축하면 기존 말뭉치에 등장하지 않았던 단어가 입력되더라도 OOV로 처리되지 않고 기존 어휘 사전을 참고해 토큰화할 수 있다.

센텐스피스 (Sentencepiece)

구글에서 개발한 오픈소스 하위 단어 토큰나이저 라이브러리. BPE와 유사한 알고리즘을 사용해 입력 데이터를 토큰화하고 단어 사전을 생성한다. 워드피스, 유니코드 기반의 다양한

알고리즘을 지원하며 사용자가 직접 설정할 수 있는 하이퍼파라미터들을 제공한다.

코포라(Korpora) 라이브러리는 국립국어원이나 AI Hub에서 제공하는 말뭉치 데이터를 쉽게 사용할 수 있게 제공하는 오픈소스 라이브러리다.

SentencePieceTrainer 주요 매개변수

매개변수	의미
<code>input</code>	말뭉치 텍스트 파일의 경로
<code>model_prefix</code>	모델 파일 이름
<code>vocab_size</code>	어휘 사전 크기
<code>character_coverage</code>	말뭉치 내에 존재하는 글자 중 토크나이저가 다룰 수 있는 글자의 비율
<code>model_type</code>	토크나이저 알고리즘 (unigram / bpe / char / word)
<code>max_sentence_length</code>	최대 문장 길이
<code>unk_id</code>	OOV를 의미하는 unk 토큰의 id (기본값: 0)
<code>bos_id</code>	문장이 시작되는 지점을 의미하는 bos 토큰의 id (기본값: 1)
<code>eos_id</code>	문장이 끝나는 지점을 의미하는 eos 토큰의 id (기본값: 2)

학습 결과 파일

파일	설명
<code>petition_bpe.model</code>	학습된 토크나이저가 저장된 파일
<code>petition_bpe.vocab</code>	어휘 사전이 저장된 파일

센텐스피스 라이브러리는 띄어쓰기나 공백도 특수문자로 취급해 토큰화 과정에서 `_` (U+2581)로 공백을 표현한다.

특수 토큰

토큰	설명
<code><unk></code>	unknown의 약자로 OOV 발생 시 매핑되는 토큰
<code><s></code>	문장의 시작 지점을 표시하는 토큰
<code></s></code>	문장의 종료 지점을 표시하는 토큰

SentencePieceProcessor 주요 메서드

메서드	설명
<code>encode_as_pieces</code>	문장을 토큰화한다

메서드	설명
<code>encode_as_ids</code>	토큰을 정수로 인코딩한다. 어휘 사전의 토큰에 매핑된 ID 값을 의미하며 이 ID 값을 활용해 자연어 처리 모델을 구축한다
<code>decode_ids</code>	정수를 문자열 데이터로 변환한다
<code>decode_pieces</code>	하위 단어 토큰을 문자열 데이터로 변환한다

워드피스 (Wordpiece)

바이트 페어 인코딩 토큰라이저와 유사한 방법으로 학습되지만, 빈도 기반이 아닌 **확률 기반**으로 글자 쌍을 병합한다.

모델이 새로운 하위 단어를 생성할 때 이전 하위 단어와 함께 나타날 확률을 계산해 가장 높은 확률을 가진 하위 단어를 선택한다.

글자 쌍 점수 수식

$$score = \frac{f(x, y)}{f(x) \cdot f(y)}$$

- `f` : 빈도(frequency)를 나타내는 함수
- `x`, `y` : 병합하려는 하위 단어
- `f(x, y)` : x와 y가 조합된 글자 쌍의 빈도
- `score` : x와 y를 병합하는 것이 적절한지를 판단하기 위한 점수

이 수식을 통해 점수가 높은 쌍부터 병합하며, 연속된 글자 쌍이 더 이상 나타나지 않거나 정해진 어휘 사전 크기에 도달할 때까지 학습한다.

토큰라이저스 (Tokenizers) 라이브러리

허깅 페이스의 토큰라이저스 라이브러리의 워드피스 API를 이용하면 쉽고 빠르게 토큰라이저를 구현하고 학습할 수 있다.

정규화 (Normalization)

일관된 형식으로 텍스트를 표준화하고 모호한 경우를 방지하기 위해 일부 문자를 대체하거나 제거하는 등의 작업을 수행한다.

클래스	설명
<code>NFD()</code>	NFD 유니코드 정규화

클래스	설명
<code>NFKD()</code>	NFKD 유니코드 정규화
<code>NFC()</code>	NFC 유니코드 정규화
<code>NFKC()</code>	NFKC 유니코드 정규화
<code>Lowercase()</code>	입력 문장의 모든 대문자를 소문자로 변환
<code>Strip()</code>	입력 문장의 양 끝에 있는 공백 제거
<code>StripAccents()</code>	모든 액센트 기호 제거
<code>Replace()</code>	문자 혹은 정규 표현식을 기반으로 입력 문장 변환
<code>BertNormalizer()</code>	BERT 모델에 사용된 정규화 적용
<code>Sequence()</code>	여러 개의 정규화 모듈을 병합해 순서대로 수행

사전 토큰화 (Pre-tokenization)

입력 문장을 토큰화하기 전에 단어와 같은 작은 단위로 나누는 기능을 제공한다.

클래스	설명
<code>Sequence()</code>	여러 개의 사전 토큰화 모듈을 병합하여 순서대로 수행
<code>ByteLevel()</code>	OpenAI에서 GPT-2를 학습할 때 사용한 방법으로 문자열을 그래픽 문자열로 매핑하고 공백을 기준으로 나눈다
<code>CharDelimiterSplit()</code>	문자 기준으로 문자열을 나눈다
<code>Digits()</code>	모든 형태의 숫자 표현을 기준으로 문자열을 나눈다
<code>Punctuation()</code>	구두점을 기준으로 입력 문자열을 나눈다
<code>Whitespace()</code>	단어 경계(공백과 구두점)를 기준으로 사전 토큰화를 진행한다
<code>WhitespaceSplit()</code>	공백 문자를 기준으로 입력 문자열을 나눈다
<code>Metaspace(replacement="_ ")</code>	Whitespace 기준으로 사전 토큰화를 진행하고, replacement로 Whitespace를 치환한다
<code>Split(pattern, behavior)</code>	정규표현식(pattern)과 동작(behavior)을 기준으로 문자열을 나눈다

Tokenizer 주요 메서드

메서드	설명
<code>encode</code>	문장을 토큰화한다
<code>encode_batch</code>	여러 문장을 한 번에 토큰화한다
<code>decode</code>	정수 인코딩된 결과를 다시 문장으로 디코딩한다

Encoding 객체 주요 속성

속성	설명
<code>tokens</code>	토큰화된 값을 확인한다
<code>ids</code>	인코딩된 문장의 ID 값을 출력한다

6장 임베딩 (Embedding)

토큰화만으로는 모델을 학습할 수 없다. 컴퓨터는 텍스트 자체를 이해할 수 없으므로 텍스트를 숫자로 변환하는 **텍스트 벡터화(Text Vectorization)** 과정이 필요하다.

1. 원-핫 인코딩 (One-Hot Encoding)

문서에 등장하는 각 단어를 고유한 색인 값으로 매핑한 후, 해당 색인 위치를 1로 표시하고 나머지 위치는 모두 0으로 표시하는 방식이다.

색인	0	1	2	3
토큰	I	like	apples	bananas

- I like apples: [1, 1, 1, 0]
- I like bananas: [1, 1, 0, 1]

단점

- 벡터의 **희소성(Sparsity)** 이 크다
- 말뭉치 내에 존재하는 토큰의 개수만큼의 벡터 차원이 필요하지만 입력 문장 내 토큰 수는 현저히 적어 컴퓨팅 비용 증가와 차원의 저주 문제가 발생한다
- 텍스트의 벡터가 입력 텍스트의 의미를 내포하지 않으므로 두 문장이 의미적으로 유사해도 벡터가 유사하게 나타나지 않을 수 있다

2. 빈도 벡터화 (Count Vectorization)

문서에서 단어의 빈도수를 세어 해당 단어의 빈도를 벡터로 표현하는 방식이다.

원-핫 인코딩 방식은 같은 단어가 여러 번 등장하더라도 1과 0으로 표현하지만, 빈도 벡터화는 해당 단어의 빈도로 표시한다.

3. 워드 임베딩 (Word Embedding)

단어를 고정된 길이의 실수 벡터로 표현하는 방법. 단어의 의미를 벡터 공간에서 다른 단어와의 상대적 위치로 표현해 단어 간의 관계를 추론한다.

대표 모델: Word2Vec, fastText

단점

- 고정된 임베딩을 학습하기 때문에 다의어나 문맥 정보를 다루기 어렵다

동적 임베딩 (Dynamic Embedding)

인공 신경망을 활용해 문맥에 따라 다르게 표현되는 임베딩 기법.

대표 모델: GPT, BERT

4. 언어 모델 (Language Model)

입력된 문장으로 각 문장을 생성할 수 있는 확률을 계산하는 모델. 주어진 문장을 바탕으로 문맥을 이해하고, 문장 구성에 대한 예측을 수행한다.

자동 번역, 음성 인식, 텍스트 요약 등 다양한 자연어 처리 분야에서 활용된다.

완성된 문장 단위로 확률을 계산하는 것은 불가능에 가깝기 때문에, 문장 전체를 예측하는 방법 대신 하나의 토큰 단위로 예측하는 방법인 **자기회귀 언어 모델** 이 고안됐다.

자기회귀 언어 모델 (Autoregressive Language Model)

입력된 문장들의 조건부 확률을 이용해 다음에 올 단어를 예측한다. 이전에 등장한 모든 토큰의 정보를 고려하며, 문장의 문맥 정보를 파악하여 다음 단어를 생성한다. 다음 단어는 다시 이전 단어를 기반으로 예측이 이루어지며, 이 과정이 반복된다.

모델의 출력값이 다시 모델의 입력값으로 사용되는 특징 때문에 **자기회귀** 라는 이름이 붙었다.

자기회귀 언어 모델은 시점별로 다음에 올 토큰을 예측하는 것이므로, **토큰 분류 문제** 로 정의할 수 있다.

조건부 확률의 연쇄법칙 (Chain rule for conditional probability)

하나의 사건이 일어날 확률을 다른 사건들의 조건부 확률을 이용해 계산하는 방식. 이전 단어들의 시퀀스가 주어졌을 때, 다음에 등장하는 단어의 확률을 이전 단어들의 조건부 확률을 이용해 계산한다.

문장 전체의 확률은 첫 번째 단어의 확률 $P(w_1)$ 과 각 단어가 이전 단어들의 조건부 확률에 따라 발생할 확률의 곱으로 나타낼 수 있다.

통계적 언어 모델 (Statistical Language Model)

언어의 통계적 구조를 이용해 문장이나 단어의 시퀀스를 생성하거나 분석한다. 시퀀스에 대한 확률 분포를 추정해 문장의 문맥을 파악해 다음에 등장할 단어의 확률을 예측한다.

일반적으로 **마르코프 체인(Markov Chain)** 을 이용해 구현된다. 마르코프 체인은 빈도 기반의 조건부 확률 모델 중 하나로 이전 상태와 현재 상태 간의 전이 확률을 이용해 다음 상태를 예측한다.

단점

- 단어의 순서와 빈도에만 기초해 문장의 확률을 예측하므로 문맥을 제대로 파악하지 못하면 불완전하거나 부적절한 결과를 생성할 수 있다
- 한 번도 등장한 적이 없는 단어나 문장에 대해서는 정확한 확률을 예측하기가 어렵다 → **데이터 희소성(Data sparsity)** 문제

장점

- 기존에 학습한 텍스트 데이터에서 패턴을 찾아 확률 분포를 생성하므로 새로운 문장을 생성할 수 있다
- 대규모 자연어 데이터를 처리하는 데 효과적이다

GPT와 BERT의 사전 학습 방식

모델	방식	설명
GPT (Generative Pre-trained Transformer)	인과적 언어 모델	앞선 단어를 기반으로 다음 단어를 예측한다
BERT (Bidirectional Encoder Representations from Transformers)	마스킹된 언어 모델	문장에서 마스킹된 단어를 양방향으로 예측한다

N-gram

가장 기초적인 통계적 언어 모델. 텍스트에서 N개의 연속된 단어 시퀀스를 하나의 단위로 취급하여 특정 단어 시퀀스가 등장할 확률을 추정한다.

N-gram 모델은 입력 텍스트를 하나의 토큰 단위로 분석하지 않고 N개의 토큰을 묶어서 분석한다.

N-gram 종류

N값	명칭	설명
N = 1	유니그램 (Unigram)	단일 토큰 단위
N = 2	바이그램 (Bigram)	2개의 연속된 토큰 단위
N = 3	트라이그램 (Trigram)	3개의 연속된 토큰 단위
N ≥ 4	N-gram	N개의 연속된 토큰 단위

N-gram 언어 모델은 모든 토큰을 사용하지 않고 N-1개의 토큰만을 고려해 확률을 계산한다.

활용 분야

- 소규모 데이터셋에서 연속된 문자열 패턴 분석
- 관용적 표현 분석 (예: '입이 무겁다' → '비밀을 잘 지킨다')
- 단어의 순서가 중요한 자연어 처리 작업 및 문자열 패턴 분석

5. TF-IDF

TF-IDF(Term Frequency-Inverse Document Frequency) 란 텍스트 문서에서 특정 단어의 중요도를 계산하는 방법으로, 문서 내에서 단어의 중요도를 평가하는 데 사용되는 통계적인 가중치를 의미한다. **BoW(Bag-of-Words)** 에 가중치를 부여하는 방법이다.

BoW (Bag-of-Words)

문서나 문장을 단어의 집합으로 표현하는 방법으로, 문서나 문장에 등장하는 단어의 중복을 허용해 빈도를 기록한다.

BoW를 이용해 벡터화하는 경우 모든 단어는 동일한 가중치를 갖는다는 단점이 있다.

TF (단어 빈도, Term Frequency)

문서 내에서 특정 단어의 빈도수를 나타내는 값이다.

$$TF(t, d) = count(t, d)$$

TF 값이 높을수록 해당 단어가 특정 문서에서 중요한 역할을 한다고 볼 수 있다. 하지만 문서의 길이가 길어질수록 해당 단어의 TF 값도 높아질 수 있다.

DF (문서 빈도, Document Frequency)

한 단어가 얼마나 많은 문서에 나타나는지를 의미한다.

$$DF(t, D) = count(t \in d : d \in D)$$

- DF 값이 높으면 → 많은 문서에 등장 → 널리 사용되는 단어 → 중요도 낮을 수 있음
- DF 값이 낮으면 → 적은 문서에만 등장 → 특정 문맥에서만 사용 → 중요도 높을 수 있음

IDF (역문서 빈도, Inverse Document Frequency)

전체 문서 수를 문서 빈도로 나눈 다음에 로그를 취한 값. 문서 내에서 특정 단어가 얼마나 중요한지를 나타낸다.

$$IDF(t, D) = \log \left(\frac{\text{count}(D)}{1 + DF(t, D)} \right)$$

- 분모에 1을 더하는 이유: 특정 단어가 한 번도 등장하지 않을 때 분모가 0이 되는 것을 방지
- 로그를 취하는 이유: 너무 큰 값이 나오는 것을 방지하고 정교한 가중치를 얻기 위함

TF-IDF 계산

$$TF-IDF(t, d, D) = TF(t, d) \times IDF(t, D)$$

문서 내에 단어가 자주 등장하지만, 전체 문서 내에 해당 단어가 적게 등장한다면 TF-IDF 값은 커진다. 전체 문서에서 자주 등장할 확률이 높은 관사나 관용어 등의 가중치는 낮아진다.

단점

- 문장의 순서나 문맥을 고려하지 않는다
- 벡터가 해당 문서 내의 중요도를 의미할 뿐, 벡터가 단어의 의미를 담고 있지 않다

사이킷런 TF-IDF 주요 매개변수

매개변수	설명
<code>input</code>	입력될 데이터의 형태. 기본값 <code>content</code> (문자열/바이트), <code>file</code> (파일 객체), <code>filename</code> (파일 경로)
<code>encoding</code>	바이트 혹은 파일을 입력값으로 받을 경우 사용할 텍스트 인코딩 값
<code>lowercase</code>	입력받은 데이터를 소문자로 변환할지 여부. 참값으로 설정하면 모든 입력 텍스트를 소문자로 변환한다
<code>stop_words</code>	분석에 도움이 되지 않는 의미 없는 단어들. 입력받은 단어들은 단어 사전에 추가되지 않는다
<code>ngram_range</code>	사용할 N-gram의 범위. (최솟값, 최댓값) 형태로 입력. (1,1)은 유니그램, (1,2)는 유니그램+바이그램
<code>max_df</code>	전체 문서 중 일정 횟수 이상 등장한 단어는 불용어로 처리. 정수=등장 횟수 초과, 실수(1 이하)=비율 초과
<code>min_df</code>	전체 문서 중 일정 횟수 미만으로 등장한 단어를 불용어로 처리
<code>vocabulary</code>	미리 구축한 단어사전이 있다면 해당 단어 사전을 사용. 미입력 시 TF-IDF 학습 시 자동으로 구축

매개변수	설명
<code>smooth_idf</code>	IDF 계산 시 분모에 1을 더한다

주요 메서드 및 속성

이름	유형	설명
<code>fit</code>	메서드	말뭉치를 학습한다
<code>transform</code>	메서드	데이터 변환을 수행한다
<code>fit_transform</code>	메서드	학습과 변환을 동시에 수행한다
<code>toarray</code>	메서드	TF-IDF를 넘파이 배열로 변환한다. (문서 수) × (단어 수) 형태
<code>vocabulary_</code>	속성	TF-IDF에 사용된 단어 사전. 키=단어, 값=색인 값

6. Word2Vec

2013년 구글에서 공개한 임베딩 모델로 단어 간의 유사성을 측정하기 위해 **분포 가설 (distributional hypothesis)** 을 기반으로 개발됐다.

분포 가설 (Distributional Hypothesis)

같은 문맥에서 함께 자주 나타나는 단어들은 서로 유사한 의미를 가질 가능성이 높다는 가정이다.

분포 가설은 단어 간의 **동시 발생(co-occurrence)** 확률 분포를 이용해 단어 간의 유사성을 측정한다.

정리 비교표

토큰화 방법 비교

방법	단위	장점	단점
단어 토큰화	단어	간단, 직관적	OOV, 조사·부호에 취약
글자 토큰화	글자/자소	작은 사전, OOV 감소	의미 없는 개별 토큰, 시퀀스 길이 증가
형태소 토큰화	형태소	문법 구조 반영, 한국어 적합	신조어·외래어 취약

방법	단위	장점	단점
하위 단어 토큰화 (BPE)	하위 단어	OOV 완화, 빈도 기반 병합	의미보다 빈도 위주
하위 단어 토큰화 (Wordpiece)	하위 단어	확률 기반, 정확한 분리	학습 비용 존재

임베딩 방법 비교

방법	장점	단점
원-핫 인코딩	단순, 구현 쉬움	희소성 큼, 의미 미반영
빈도 벡터화	빈도 반영	순서·문맥 미반영
TF-IDF	중요도 가중치 부여	순서·문맥 미반영, 의미 미반영
워드 임베딩 (Word2Vec)	의미를 벡터로 표현	다의어·문맥 처리 어려움
동적 임베딩 (GPT, BERT)	문맥 반영, 높은 성능	대규모 리소스 필요